→ FIRST(X) = { FIRST($Y_1$) ∪ FIRST($Y_2$) ∪ . . . ∪ FIRST($Y_k$) }
where $Y_1 \cdots Y_k \overset{*}{\Rightarrow} \epsilon$.

- For example, everything in FIRST($Y_1$) is surely in FIRST(X), if $Y_1$ does not derive $\epsilon$, then we add nothing more to FIRST(X), but if $Y_1 \overset{*}{\Rightarrow} \epsilon$, then we add FIRST($Y_2$), and so on.

* FOLLOW function :

→ FOLLOW(A) is defined as the set of terminal symbols that appear immediately to the right of A.

→ The rules for computing FOLLOW function are as given below :

1) Place "$" in FOLLOW(S), where S is start symbol, and $ is the input right endmarker.

2) If there is a production $A \rightarrow \alpha B \beta$, then everything in FIRST($\beta$) except $\epsilon$ is in FOLLOW(B).

3) If there is a production $A \rightarrow \alpha B \beta$, or a production $A \rightarrow \alpha B$, where FIRST($\beta$) = {$\epsilon$}, then everything in FOLLOW(A) is in FOLLOW(B).

* Example :

→ Consider grammar :

$E \rightarrow TE'$
$E' \rightarrow +TE' \mid \epsilon$
$T \rightarrow FT'$
$T' \rightarrow *FT' \mid \epsilon$
$F \rightarrow (E) \mid id$

find the FIRST and FOLLOW functions for the above grammar.

→ FIRST function :

FIRST(E) = FIRST(T) = FIRST(F)

as $F \rightarrow (E) | id$

* FIRST(E) = FIRST(T) = FIRST(F) = {(, id}

* FIRST(E') = {+, ∈}

* FIRST(T') = {*, ∈}.

→ FOLLOW(" function :

* FOLLOW(E) :

– Since E is a start symbol, add $ to follow of E.

– FOLLOW(E) = FIRST( ) ) = { ) }. [∵ rule 2 – follow]

Hence FOLLOW(E) = { ), $}.

* FOLLOW(E') :

$E \rightarrow TE'$:

i) ∧ FOLLOW(E') = FOLLOW(E) [∵ Follow – rule 3]

= { ), $)

ii) $E' \rightarrow +TE'$ :

FOLLOW(E') = FOLLOW(E') [∵ rule 3 – FOLLOW]

Hence, FOLLOW(E') = { ), $}.

* FOLLOW(T) :

i) $E \rightarrow TE'$:

FOLLOW(T) = {FIRST(E') – ∈} [∵ rule 2 – FOLLOW]

= {(+, ∈} – ∈}

= {+}.

ii) $E' \rightarrow +TE'$ :

FOLLOW(T) = FOLLOW(E') [∵ rule 3 – FOLLOW]

= { ), $}.

Finally FOLLOW(T) = {+, ), $}.

* FOLLOW($T^1$):

i) $T \rightarrow FT^1$:

FOLLOW($T^1$) = FOLLOW($T$) [∵ rule3 - FOLLOW]

= {+, ), $}

ii) $T^1 \rightarrow *FT^1$

FOLLOW($T^1$) = FOLLOW($T^1$) [∵ rule3 - FOLLOW]

Hence FOLLOW($T^1$) = {+, ), $}.

* FOLLOW($F$):

i) $T \rightarrow FT^1$, $T^1 \rightarrow *FT^1 | \epsilon$

FOLLOW($F$) = {FIRST($T^1$) - $\epsilon$} [∵ rule 2]

= {(*, $\epsilon$) - $\epsilon$}

= {*}

ii) FOLLOW($F$) = FOLLOW($T^1$) [∵ rule-3]

= {+, ), $}

Finally, FOLLOW($F$) = {+, *, ), $}.

| Symbols | FIRST | FOLLOW |
|---|---|---|
| E | {(, id} | {), $} |
| $E^1$ | {+, $\epsilon$} | {), $} |
| T | {(, id} | {+, ), $} |
| $T^1$ | {*, $\epsilon$} | {+, ), $} |
| F | {(, id} | {+, *, ), $} |

* Algorithm: construction of Predictive Parsing Table

Input: Grammar G

output: Parsing Table M.

Method:

For each production $A \to \alpha$ of the grammar, do the following:

1) For each terminal $a$ in FIRST($\alpha$), add $A \to \alpha$ to M[A, a].

2) If $\epsilon$ is in FIRST($\alpha$), then for each terminal $b$ in FOLLOW(A), add $A \to \alpha$ to M[A, b].

- If $\epsilon$ is in FIRST($\alpha$) and \$ is in FOLLOW(A), add $A \to \alpha$ to M[A, \$] as well.

3) If, after performing the above, there is no production at all in M[A, a], then set M[A, a] to error. (which we normally represent by an empty entry in the table).

* Example:

Grammar:

$E \to TE'$

$E' \to +TE' | \epsilon$

$T \to FT'$

$T' \to *FT' | \epsilon$

$F \to (E) | id.$

- consider each rule one by one to fill up the entries in the Parsing Table.

i) $E \to TE'$

FIRST($TE'$) = FIRST($T$) = {(, id}

$\therefore$ M[E, (] = $E \to TE'$

M[E, id] = $E \to TE'$

ii) $E' \to +TE'$

FIRST($+TE'$) = {+}

$\therefore$ M[E', +] = $E' \to +TE'$

* $E' \to \epsilon$

FOLLOW($E'$) = {), \$}

M[E', )] = $E' \to \epsilon$

M[E', \$] = $E' \to \epsilon$

iii) $T \rightarrow FT'$

$FIRST(FT') = FIRST(F) = \{(, id\}$

Hence $M[T, (] = T \rightarrow FT'$

$M[T, id] = T \rightarrow FT'$

iv) $T' \rightarrow *FT'$

$FIRST(*FT') = \{*\}$

$M[T', *] = T' \rightarrow *FT'$

* $T' \rightarrow \epsilon$

$FIRST(\epsilon) = \{\epsilon\}$

Hence $FOLLOW(T') = \{+, ), \$\}$

$M[T', +] = T' \rightarrow \epsilon$

$M[T', )] = T' \rightarrow \epsilon$

$M[T', \$] = T' \rightarrow \epsilon$.

v) $F \rightarrow (E)$

$FIRST((E)) = \{(\}$

$M[F, (] = F \rightarrow (E)$

* $F \rightarrow id$

$FIRST(F) = \{id\}$

$M[F, id] = F \rightarrow id$.

| Non-terminal | Input symbol | | | | | |
|---|---|---|---|---|---|---|
| | id | + | * | ( | ) | $ |
| E | $E \rightarrow TE'$ | | | $E \rightarrow TE'$ | | |
| $E'$ | | $E' \rightarrow +TE'$ | | | $E' \rightarrow \epsilon$ | $E' \rightarrow \epsilon$ |
| T | $T \rightarrow FT'$ | | | $T \rightarrow FT'$ | | |
| $T'$ | | $T' \rightarrow \epsilon$ | $T' \rightarrow *FT'$ | | $T' \rightarrow \epsilon$ | $T' \rightarrow \epsilon$ |
| F | $F \rightarrow id$ | | | $F \rightarrow (E)$ | | |

Fig. Parsing Table M

* Parsing the input string:

Example:

input string: id+id*id

→ At the initial configuration the stack will contain start symbol E, in the input buffer input buffer is placed.

| Stack | Input | Action |
|---|---|---|
| $E | id+id*id$ | |

→ Now symbol E is at top of the stack and input pointer is at first id, Hence M[E,id] is referred i.e, $E \to TE'$, so we will push $E'$ first then T.

| Stack | Input | Action |
|---|---|---|
| $E | id+id*id$ | |
| $E'T | id+id*id$ | output $E \to TE'$ |
| $E'T'F | id+id*id$ | output $T \to FT'$ |
| $E'T'id | id+id*id$ | output $F \to id$ |
| $E'T' | +id*id$ | match id. |
| $E' | +id*id$ | output $T' \to \epsilon$ |
| $E'T+ | +id*id$ | output $E' \to +TE'$ |
| $E'T | id*id$ | match +. |
| $E'T'F | id*id$ | output $T \to FT'$ |
| $E'T'id | id*id$ | output $F \to id$ |
| $E'T' | *id$ | match id. |
| $E'T'F* | *id$ | output $T' \to *FT'$ |
| $E'T'F | id$ | match * |
| $E'T'id | id$ | output $F \to id$ |
| $E'T' | $ | match id. |
| $E' | $ | output $T' \to \epsilon$ |
| $ | $ | output $E' \to \epsilon$. |

Fig. Moves made by a predictive parser on input: id+id*id.